Průchody grafem

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

21.10.2026

CC BY-NC-SA 4.0

Motivace

Přátelé přátel

LinkedIn u každého člověka ukazuje **stupeň známosti**: „1.“ je váš přítel, „2.“ přítel přítele, „3.“ ještě o krok dál

Jak byste stupeň známosti spočítali?

Lidé jsou **vrcholy**, známosti jsou **hrany** a stupeň známosti je **vzdálenost** v grafu

Projdeme své přátele, pak **jejich** přátele, pak přátele jejich přátel… Graf tedy musíme **systematicky projít**

Od uložení k průchodu

Minule jsme graf **uložili** do paměti: maticí sousednosti, nebo seznamem sousedů

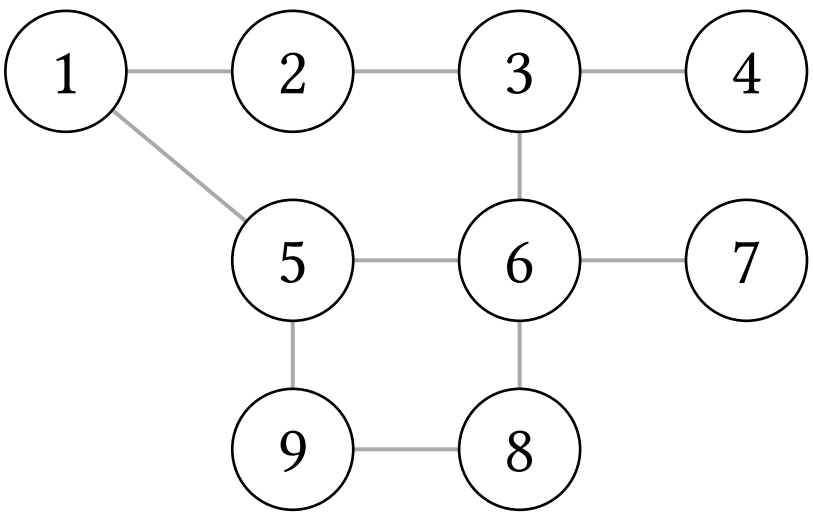
Dnes ho budeme **procházet**, tedy každý vrchol navštívíme **právě jednou**

U stromů to už umíme: inorder, preorder, postorder

Proč to pro graf nestačí?

- Graf **nemá kořen**, takže si musíme vybrat startovní vrchol
- Graf může mít **kružnice**, po kterých bychom chodili dokola

Příklad grafu



Graf z minulé přednášky, tentokrát **neorientovaný**

Co si musíme během průchodu pamatovat?

1. Které vrcholy jsme už **viděli**
2. Které **čekají** – v **zásobníku**, nebo ve **frontě**

Zásobník a fronta

Zásobník

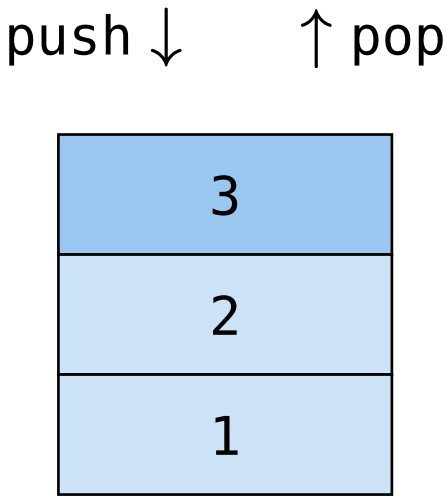
Zásobník už známe. **Kde jsme ho viděli?**

- **Zásobník volání** (*call stack*) – rámce volaných funkcí (přednášky o segmentaci paměti a rekurzi)
- Analogie: **komínek talířů** – bereme vždy ten, který jsme položili naposledy

Zásobník

Princip **LIFO** (*Last In, First Out*)

- `push` – vloží prvek **navrch**
- `pop` – odebere prvek **shora**
- `top` – přečte horní prvek

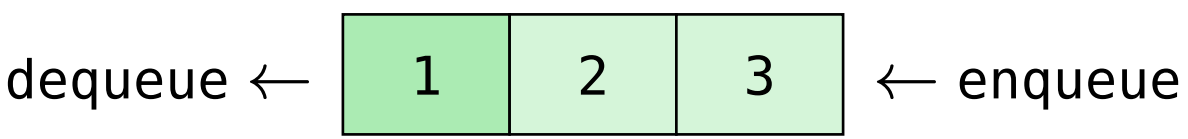


Fronta

Co si představíte pod pojmem fronta?

Fronta v jídelně – kdo přijde **první**, je **první** obsloužen: princip **FIFO** (*First In, First Out*)

- `enqueue` – vloží prvek na **konec**
- `dequeue` – odebere prvek ze **začátku**
- `front` – přečte prvek na začátku



Zásobník vs. fronta

Vložíme `1`, `2`, `3` a pak třikrát prvek vyjmeme. **Co dostaneme?**

Zásobník:

`3 2 1`

Fronta:

`1 2 3`

Zásobník pořadí **obrací**, fronta ho **zachovává**

Implementace zásobníku

Zásobník pomocí pole

Jak byste zásobník postavili z pole? Stačí pole a index `top`, který udává počet prvků

- `push` – zapíšeme prvek na index `top` a `top` zvýšíme
- `pop` – `top` snížíme a vrátíme prvek na indexu `top`

Složitost obou operací: $\mathcal{O}(1)$

Problém: pevná kapacita, kterou vyřeší **dynamické pole** z přednášky o amortizované složitosti – složitost `push`: $\mathcal{O}^*(1)$

Zásobník pomocí spojového seznamu

Jak byste zásobník postavili ze spojového seznamu?

- `push` – vložíme prvek na **začátek** seznamu (`prepend`)
- `pop` – smažeme **první** prvek

Složitost obou operací: $\mathcal{O}(1)$ a to **vždy**, ne jen amortizovaně

Cena: každý prvek potřebuje vlastní `malloc` a pointer `next` navíc

Implementace fronty

Fronta pomocí pole

Naivně: dequeue vezme `queue[0]` a zbytek posune doleva – složitost dequeue: $\mathcal{O}(n)$

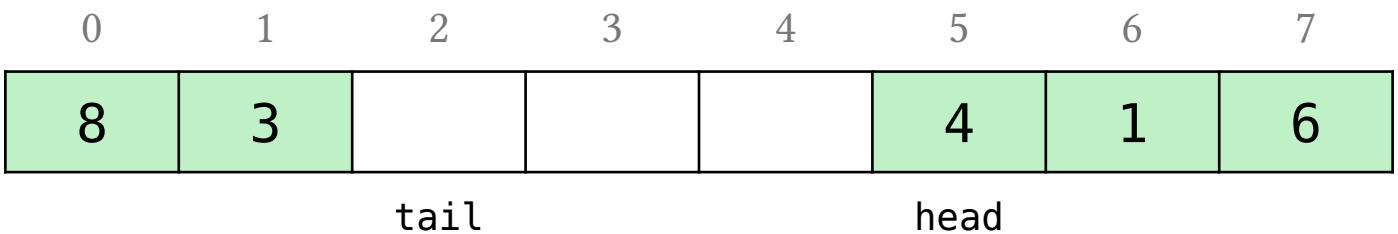
Lépe: nic neposouváme, jen si pamatujeme index začátku (`head`) a počet prvků

Co se stane, když fronta dojde na konec pole? Na začátku pole je přitom volné místo…

Řešením je **kruhová fronta** (*ring buffer*), která za posledním indexem pokračuje opět od nuly

Fronta pomocí pole

Kruhová fronta s kapacitou 8:



Co je ve frontě? `4 1 6 8 3`, protože se „přetočila“ přes konec

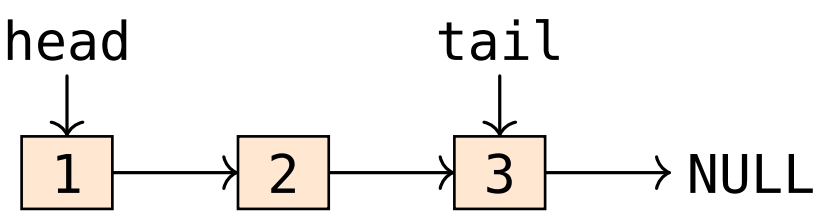
Kam zapíše další enqueue? `(head + count) % 8` $= (5 + 5) \bmod 8 = 2$

Zajímavost: kruhovou frontu najdete např. v jádře Linuxu (`kfifo`)

Fronta pomocí spojového seznamu

- `dequeue` – smažeme **první** prvek – složitost: $\mathcal{O}(1)$
- `enqueue` – vložíme prvek na **konec** – složitost: $\mathcal{O}(n)$, protože musíme projít celý seznam

Řešení: pamatujeme si i pointer na **poslední** prvek – složitost obou operací: $\mathcal{O}(1)$



Srovnání implementací

	Pole	Spojový seznam
Vložení / odebrání	$\mathcal{O}(1)$, při zvětšování $\mathcal{O}^*(1)$	$\mathcal{O}(1)$
Paměť navíc	nevyužitá kapacita	pointer a `malloc` u každého prvku
Rozložení v paměti	prvky vedle sebe	prvky rozházené po haldě
Maximální velikost	kapacita (nebo `realloc`)	jen dostupná paměť

V praxi častěji vyhrává **pole**, protože prvky leží vedle sebe a procesor je díky cache čte rychleji

Průchod do šířky

Princip BFS

Průchod do šířky (*Breadth-First Search*, BFS) prochází graf **po vrstvách**: nejdřív všechny sousedy startovního vrcholu, pak sousedy sousedů, …

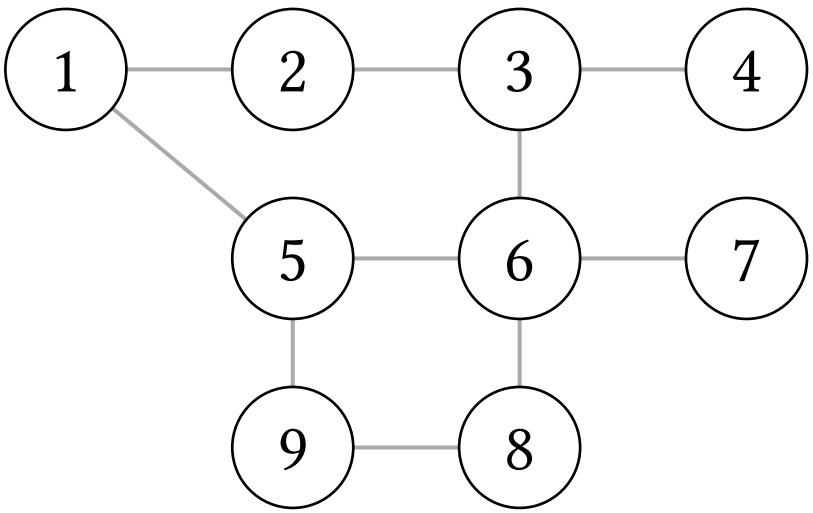
Jakou analogii si můžete představit? Kruhy na vodě, když do ní hodíte kámen

Princip BFS

Princip:

1. Startovní vrchol **označíme** a vložíme do **fronty**
2. Dokud fronta není prázdná, vyjmeme z ní vrchol v
3. Všechny dosud neoznačené sousedy vrcholu v označíme a vložíme na konec fronty

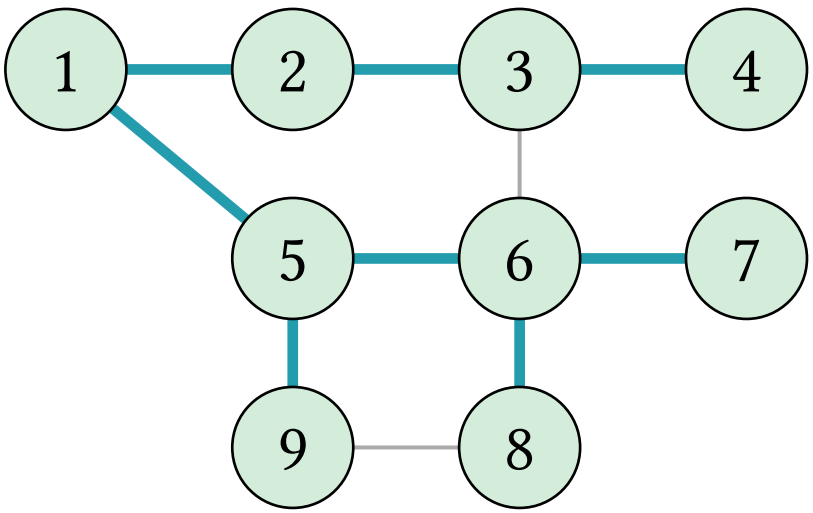
Předpověď pořadí



V jakém pořadí BFS z vrcholu 1 vrcholy navštíví?

Sousedy bereme vždy **od nejmenšího** čísla

Animace BFS



Fronta (začátek vlevo):

prázdná

Výstup: 1 2 5 3 6 9 4 7 8 ✓

aktuální ve frontě hotovo

Vyjmeme **4**, **7** a **8** → žádní noví sousedé, fronta je prázdná → **hotovo!**

Rozbor BFS

Co se stalo při zpracování vrcholu 3?

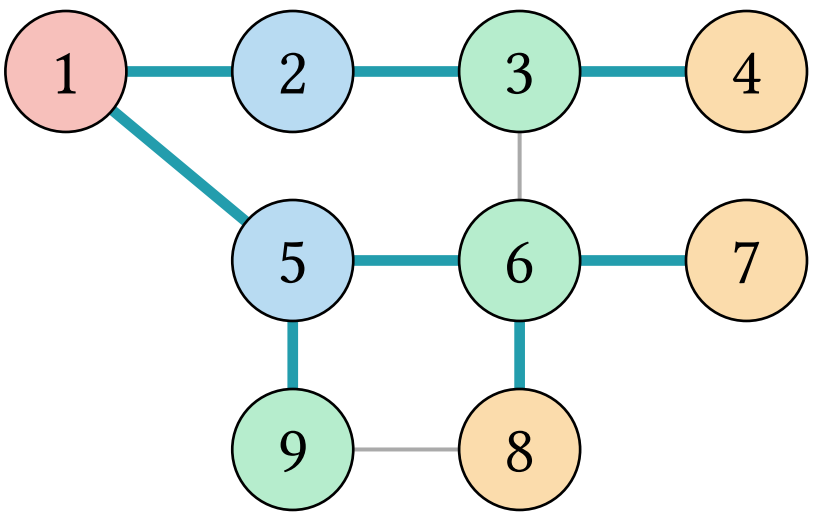
Vrchol **6** je sousedem **3**, ale už čekal ve frontě

Proto vrchol **označujeme už při vložení** do fronty, ne až při vyjmutí

Co by se stalo, kdybychom označovali až při vyjmutí?

Vrchol **6** by se do fronty dostal **dvakrát** (od **5** i od **3**) a zpracovali bychom ho dvakrát

Vrstvy BFS



1 | 2 5 | 3 6 9 | 4 7 8

vrstvy podle vzdálenosti od **1**

Cesta po modrých hranách (**BFS strom**) je **nejkratší** možná

Právě proto BFS vymyslel Edward F. Moore (1959), který hledal nejkratší cestu bludištěm

LinkedIn „2.“ je přesně vrstva **2**!

Průchod do hloubky

Princip DFS

Průchod do hloubky (*Depth-First Search*, DFS) jde z vrcholu **co nejhlouběji** a teprve ve slepé uličce se vrací zpět

Jakou analogii si můžete představit? Bludiště: jdeme pořád dál a ve slepé uličce se vrátíme k poslední odbočce

Tuto strategii pro bludiště popsal Charles Pierre Trémaux už v 19. století

Princip DFS

Princip (rekurzivně):

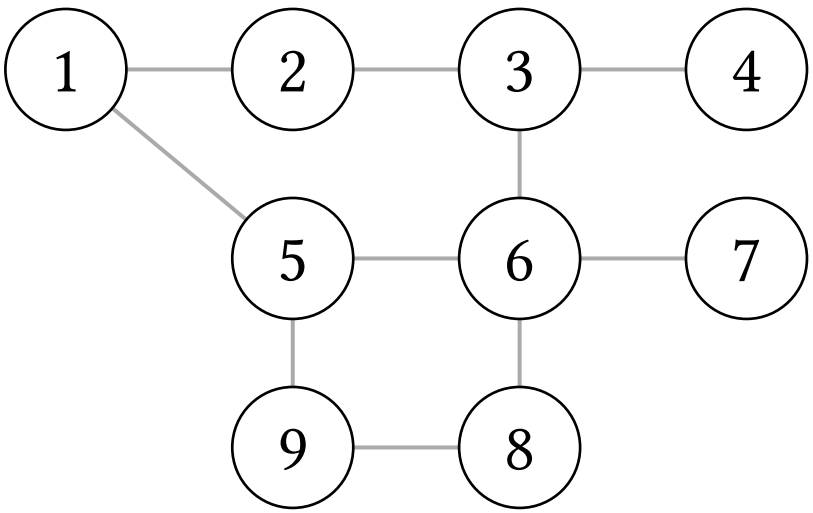
1. Vrchol v **označíme** jako navštívený
2. Pro každého nenavštíveného souseda u zavoláme DFS(u)

Kde je v DFS ten zásobník?

Je to **zásobník volání**! Každé volání `dfs(u)` na něj přidá nový rámec a když skončí, vrátíme se přesně do vrcholu, ze kterého jsme přišli

Rekurze si cestu zpět pamatuje **za nás**, stejně jako u výpisu spojového seznamu od konce v přednášce o rekurzi

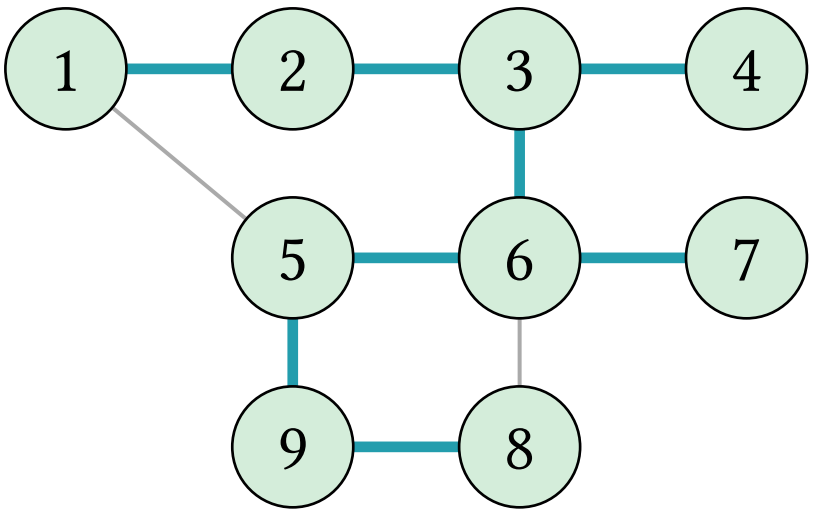
Předpověď pořadí



V jakém pořadí DFS z vrcholu 1 vrcholy navštíví?

Sousedy bereme opět **od nejmenšího** čísla

Animace DFS



Zásobník volání:

Výstup: 1 2 3 4 6 5 9 8 7 ✓

aktuální na zásobníku hotovo

7 je slepá ulička → vracíme se až do 1 → **hotovo!**

Rozbor DFS

Co se stalo při zpracování vrcholu 8?

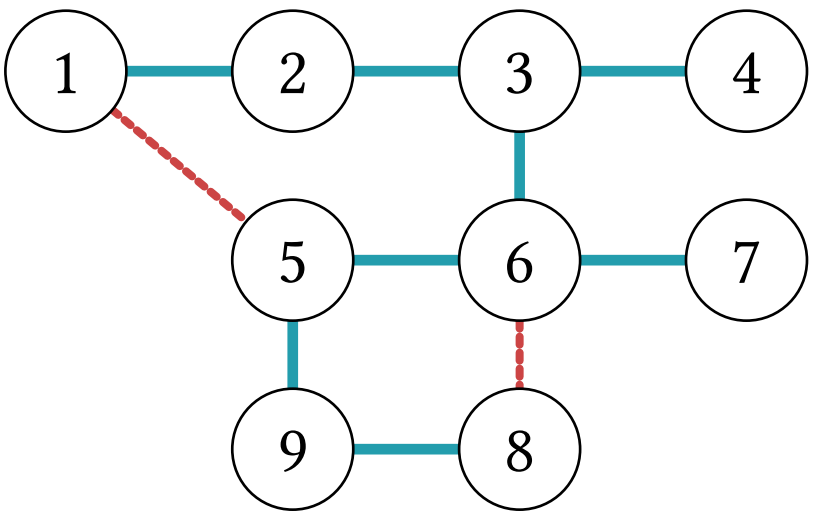
Z **8** už nebylo kam jít, takže skončila volání `dfs(8)`, `dfs(9)` i `dfs(5)` a vrátili jsme se až do vrcholu **6**, který má ještě nenavštíveného souseda **7**

Co kdybychom zapomněli označovat navštívené vrcholy?

`dfs(1)` zavolá `dfs(2)`, to zavolá znovu `dfs(1)` a tak pořád dokola, protože každá hrana vede i zpět

Rekurze je nekonečná, což vede na **stack overflow**

DFS strom a kružnice



Modré hrany tvoří **DFS strom**

Červené hrany vedou do vrcholu, který už je **na zásobníku**, a každá proto uzavírá **kružnici**

Pomocí DFS tedy umíme **detekovat kružnice**

DFS bez rekurze

Problém: graf, který tvoří jedna cesta s milionem vrcholů potřebuje milion rámců na zásobníku volání, což vede na **stack overflow**

Řešení: vlastní zásobník na haldě, který může být **mnohem větší**

Co na zásobník vložíme? Místo rámců `dfs(u)` jen čísla vrcholů, které čekají na zpracování

DFS bez rekurze

Princip (se zásobníkem):

1. Startovní vrchol vložíme na **zásobník**
2. Dokud zásobník není prázdný, vyjmeme z něj vrchol v
3. Pokud už je v navštívený, **přeskočíme** ho
4. Jinak v označíme a na zásobník vložíme jeho nenavštívené sousedy, a to **od největšího**

Je to **skoro** princip BFS, jen místo fronty máme zásobník

DFS bez rekurze

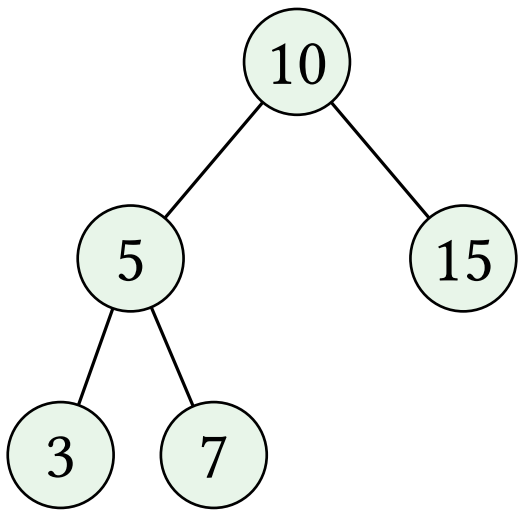
Pozor: na rozdíl od BFS označujeme vrchol až při **vyjmutí**, takže se na zásobník může dostat **víckrát** a další kopie jen přeskočíme

Proč sousedy vkládáme od největšího?

Nejmenší soused skončí na vrcholu zásobníku, takže ho vyjmeme jako první a výstup je stejný jako u rekurze: `1 2 3 4 6 5 9 8 7`

Srovnání BFS a DFS

Známé průchody stromem



DFS z kořene je **preorder**: `10 5 3 7 15`

BFS z kořene jde **po vrstvách**: `10 5 15 3 7`

Strom nemá kružnice, takže označovat navštívené vrcholy **nemusíme**

Složitost průchodů

Jakou složitost má BFS a DFS? Spočítejme práci zvlášť pro vrcholy a pro sousedy:

- **Vrcholy:** každý zpracujeme jen jednou – celkem $\mathcal{O}(|V|)$
- **Sousedé v seznamu sousedů:** dohromady $2|E|$ – celkem $\mathcal{O}(|E|)$
- **Sousedé v matici:** u každého vrcholu celý řádek – celkem $\mathcal{O}(|V|^2)$

Složitost průchodu se seznamem sousedů: $\mathcal{O}(|V| + |E|)$, s maticí: $\mathcal{O}(|V|^2)$

Složitost průchodů

Slovně: se seznamem sousedů je průchod **lineární** vzhledem k velikosti grafu

Poznámka: s maticí sousednosti – složitost průchodu je $\mathcal{O}(|V|^2)$ i pro graf s několika hranami

Kdy na tom záleží? U **řídkých** grafů: na LinkedInu má každý z milionů lidí jen stovky známostí, takže $|E| \ll |V|^2$ a seznam sousedů je **mnohem** rychlejší

BFS vs. DFS

	BFS	DFS
Datová struktura	fronta	zásobník (nebo rekurze)
Pořadí	po vrstvách	co nejhlouběji
Délka nejkratší cesty	ano	ne
Paměť navíc	velikost nejširší vrstvy	hloubka DFS stromu
Složitost	$\mathcal{O}(\|V\| + \|E\|)$	$\mathcal{O}(\|V\| + \|E\|)$

BFS a DFS v praxi

Využití BFS

- **Nejkratší cesta** v neohodnoceném grafu – stupně známosti na LinkedInu, nejmenší počet tahů v bludišti či hře
- **Návrh čipů** – C. Y. Lee (1961) nezávisle na Moorovi použil BFS k vedení spojů na desce plošných spojů
- **Web crawler** – nejdřív stáhne stránky blízko výchozí stránky, teprve potom ty vzdálené
- **Garbage collector** – Cheneyho algoritmus (1970) kopíruje živé objekty v pořadí BFS
- **Vysílání v síti** – zpráva se šíří k sousedům, pak k jejich sousedům, …

Využití DFS

- **Procházení adresářů** – `find` vypíše obsah složky dřív, než přejde na její sourozence
- `rm -r` maže jako `free_tree` z přednášky o binárních stromech: nejdřív obsah, pak složku, tedy **postorder**
- **Detekce kružnic** a pořadí závislostí (např. v `make` nebo ve správcích balíčků)
- **Řešení hádanek** – sudoku, bludiště: zkusíme tah, a když nevede k cíli, vrátíme se (*backtracking*)

Shrnutí

- **Zásobník** (LIFO) a **fronta** (FIFO) jdou postavit nad polem (kruhová fronta) i nad spojovým seznamem
- **BFS** prochází graf po vrstvách pomocí fronty a zjistí **délku nejkratší cesty** (počet hran)
- **DFS** jde co nejhlouběji pomocí zásobníku, nebo rekurze
- Kvůli kružnicím si musíme pamatovat **navštívené** vrcholy
- Se seznamem sousedů mají oba průchody složitost $\mathcal{O}(|V| + |E|)$

Co nás čeká

BFS nám řekne, jak **daleko** je každý vrchol od startu, takže umíme zjistit **délku** nejkratší cesty

Jak ale najít samotnou cestu? Tu si ukážeme v následujících přednáškách